Analytics Chat Assistant — Executive Summary

Prepared by: Amandeep Sharma
Tools Used: Google Gemini · BigQuery · Python (Flask) · Looker Studio · Docker · Cloud Run
Date: 2025
Chat link: https://analytics-chat-17391180742.us-central1.run.app
How to try:

Summary

This project demonstrates a production-grade Analytics Chat Assistant that allows business users to explore Marketing, Revenue, and Sports Performance analytics through natural language — without requiring SQL, dashboard navigation, or tool-specific knowledge.

The system combines deterministic analytics logic, LLM-powered intent parsing, metadata-driven chart discovery, and RAG-based executive explanations to replicate how a senior analytics partner would guide decision-makers through data.

This is not a chatbot demo — it is an analytics operating layer built on top of real BI infrastructure.

What This System Does (At a Glance)

- Translates natural language → filters → charts → insights
- Maintains session-aware analytical context
- Prevents hallucinations using canonical metadata + rule-based controls
- Produces executive-ready explanations, not raw data dumps
- Works across multiple analytical domains with shared architecture

Key Capabilities & Design Decisions

1) Deterministic Analytics Control (Not Prompt Guessing)

Unlike typical “chat over data” tools, this system never guesses filters, dimensions, or values.

- All campaigns, channels, clubs, products, and metrics are pre-defined canonical entities
- User input is normalized → validated → applied through strict include / exclude semantics
- Filters follow explicit rules: SET, ADD, REMOVE, CLEAR
- This guarantees auditability, reproducibility, and trust

Why this matters:
Executives will not trust analytics that "feels right but can't be explained."

2) Metadata-Driven Chart Intelligence

Charts are not hard-coded responses.

Each chart is described in BigQuery metadata:

- chart_id
- when_to_use
- primary_metrics
- caveats
- allowed_filters
- domain ownership

The assistant:

- Selects charts deterministically via concept mapping
- Falls back to fuzzy metadata scoring only when needed
- Always defaults safely to a domain master dashboard

Result:
Users get the right view for the question, not a random visualization.

3) Domain-Aware Multi-Analytics Architecture

The same assistant supports:

- Marketing analytics (campaigns, channels, CAC, efficiency)
- Revenue analytics (clubs, products, subscriptions, deal stages)
- Sports performance analytics (xG, goals, defense, seasons)

Each domain has:

- Its own filter grammar
- Its own chart vocabulary
- Its own intent logic
- Its own Looker Studio parameterization

Yet all domains share:

- The same resolver pipeline
- The same RAG engine
- The same deployment architecture

This mirrors real enterprise analytics platforms.

4) RAG for Executive Interpretation (Not Raw Data)

The RAG layer does not query fact tables.

Instead, it retrieves:

- Metric definitions
- Diagnostic patterns (e.g. “why CAC increases”)
- Known caveats
- Recommended next checks

If metadata exists → Gemini summarizes only what is known
If metadata does not exist → the system responds:

“No explanation metadata is available for this chart yet.”

No hallucination. No speculation.

This is exactly how analytics maturity is built in real organizations.

5) Session-Aware Analytical Conversation

The assistant remembers:

- Active domain
- Active filters
- Last chart
- Analytical context

Examples:

- “Why did CAC increase?” → diagnostic explanation (no chart jump)
- “Remove Facebook and show efficiency” → filters + chart update
- “Switch to sports” → domain transition + new analytical lens

This mimics how analysts actually work with stakeholders.

Strategic Use Cases

- Executive self-service analytics
- Analytics enablement for non-technical teams
- AI-assisted BI layer on top of Looker / Tableau / Power BI

- Portfolio demonstration of AI + analytics systems thinking

Technology & Architecture (High Level)

- Frontend: Flask + embedded Looker Studio
- Backend: Python, rule-based resolver, session state
- AI: Gemini (intent parsing + RAG summarization)
- Data: BigQuery (chart catalog, RAG metadata)
- Deployment: Docker + Google Cloud Run
- Security: No PII, no hard-coded secrets, environment-based keys

Final Takeaway

This project demonstrates how modern analytics products should be built:

- Not dashboards alone
- Not chatbots alone
- But controlled, explainable, domain-aware AI systems that sit between business users and data